CHAPTER 3

Neural Networks and Deep Learning

In this chapter

Introduction: The Problem That Killed AI for a Decade . . . 2
3.1 The Artificial Neuron . . . 3
3.2 Activation Functions . . . 4
3.3 Layers, Networks and Parameter Counting . . . 6
3.4 Why Depth Helps: The Feature Hierarchy . . . 7
3.5 Backpropagation: Assigning Blame Backwards . . . 7
3.6 The Training Machinery You Will Meet Inside a Transformer . . . 9
3.7 Matrices, and Why This Runs on GPUs . . . 10
3.8 Where This Sits in a Claude-Powered System . . . 11
3.9 Code Walkthrough: A Network From Scratch, With Your Own Backpropagation . . . 12
3.10 Mini Project: A Neural Language Model in 60 Lines . . . 13
3.11 Hands-on Labs . . . 16
3.12 Lab Solutions . . . 17
3.13 Best Practices . . . 17
3.14 Common Mistakes . . . 18
3.15 Performance Considerations . . . 19
3.16 Security Considerations . . . 19
3.17 Real Production Examples . . . 20
3.18 Certification Notes . . . 20
3.19 Practice Questions . . . 21
3.20 Answers and Explanations . . . 22
3.21 Ten Practical Exercises . . . 23
3.22 Five Challenge Labs . . . 24
3.23 Interview Questions . . . 24
3.24 Summary . . . 24
3.25 Glossary . . . 25
3.26 Further Reading . . . 27

NOTE

Learning Objectives

By the end of this chapter you will be able to:

1. Describe an artificial **neuron** precisely, and show that it is the logistic-regression model from Chapter 2 with one part renamed.
2. Explain what an **activation function** does, why a network without one is pointless, and choose between sigmoid, tanh, ReLU, GELU and softmax.
3. Draw a **multi-layer perceptron**, label every component, and count its parameters exactly.
4. Explain **why depth helps**: how successive layers build a hierarchy of learned features.
5. Explain **backpropagation** as an assignment of blame flowing backwards through a network — without differentiating anything by hand.
6. Name the standard training machinery you will meet inside every transformer: **Adam**, **learning-rate schedules**, **normalisation**, **dropout** and **residual connections**, and say what each one fixes.
7. Explain why deep learning runs on GPUs, in terms of **matrix multiplication**.
8. Implement a working neural network from scratch in NumPy, train it with your own backpropagation, and then build a character-level neural language model that predicts the next character — a direct, working miniature of what Claude does.

NOTE

Prerequisites

Chapters 1 and 2. You need the vocabulary of parameters, loss, gradient descent, training versus inference, overfitting and generalisation. If "the loss function scores how wrong the model is, and gradient descent nudges parameters downhill on it" does not feel obvious, re-read sections 2.1 and 2.3 before continuing.

No calculus is required. The chapter is explicit about the two places where calculus is doing the work underneath, and gives you the correct intuition instead.

Introduction: The Problem That Killed AI for a Decade

In 1958 the perceptron — a single artificial neuron — was demonstrated to a press corps that reported it as the beginning of thinking machines. In 1969 a book by Minsky and Papert showed that a single perceptron cannot learn a function as trivial as **exclusive or**.

Here is that function. Two inputs, one output, four possible cases:

A	B	A XOR B
0	0	0
0	1	1
1	0	1
1	1	0

You can verify by hand that no straight line separates the two output classes when you plot those four points. A single neuron computes a weighted sum and applies a threshold — geometrically, it draws a straight line. So no single neuron can ever solve XOR, no matter how it is trained, no matter how much data it sees.

Funding collapsed. The field entered its first winter.

The fix was known almost immediately and was, in outline, obvious: use more than one layer. A first layer draws two lines; a second layer combines them. What was *not* known was how to train such a thing. With one neuron you can compute directly how wrong it was. With a neuron three layers deep, buried behind others, there was no practical way to work out how much of the final error was its fault.

That question — *how do I assign blame to a parameter that never touches the output* — is the whole of this chapter. Its answer, **backpropagation**, took until the mid-1980s to become widely understood and until 2012 to become economically practical. Every model in this book, Claude included, is trained by it.

3.1 The Artificial Neuron

3.1.1 It is the Chapter 2 model, renamed

Recall the spam classifier from section 2.7:

```
z = w1 x1  +  w2 x2  +  w3 x3  +  b
p = sigmoid(z)
```

That is an artificial neuron. Completely. The only thing this chapter adds is that we will connect many of them together.

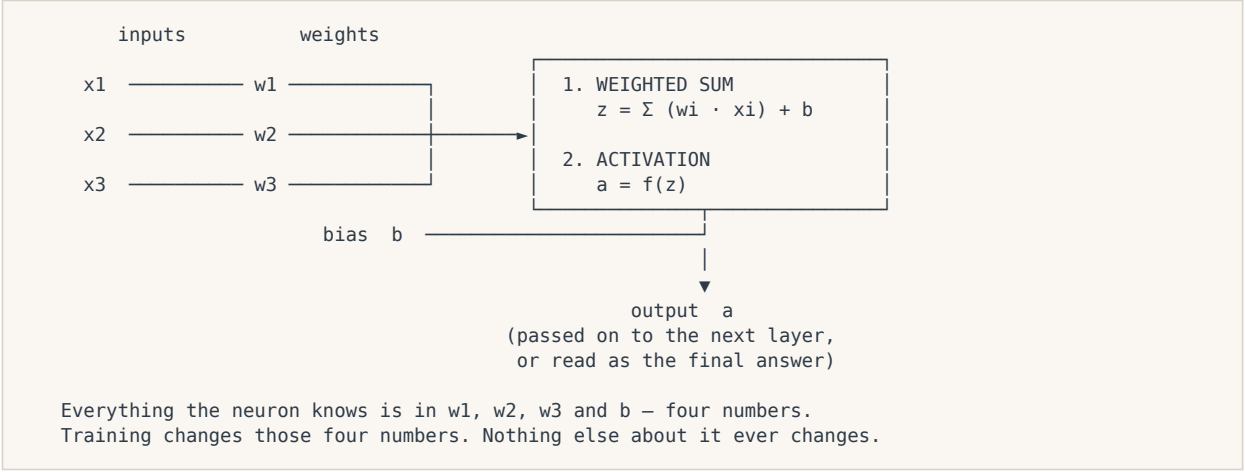


Figure 3.1 — Anatomy of a single artificial neuron.

DEFINITION

Neuron, weight, bias, activation

A **neuron** computes a weighted sum of its inputs plus a **bias**, then applies a non-linear **activation function** to the result. The **weights** determine how much each input matters; the bias shifts the threshold at which the neuron becomes active. Weights and biases are the parameters that training adjusts.

VISUAL ANALOGY

The committee vote

A neuron is a committee member deciding whether to approve a loan. Each piece of evidence — income, credit history, existing debt — is one input. The weights are how much this particular member trusts each piece of evidence. The bias is their innate disposition: a cautious member needs more positive evidence before approving.

The activation function is the rule that turns their internal tally into a decision. A harsh member has a step function: approve or refuse, nothing in between. A measured member has a sigmoid: they express a degree of confidence.

A **network** is then a hierarchy of committees. The first tier looks at raw evidence. The second tier looks not at the evidence but at what the first tier concluded. The third looks at the second. Nobody at the top ever sees the raw application — only progressively more abstract summaries of it.

Where the analogy breaks: committee members know what their inputs mean. A neuron in layer four has no interpretable meaning assigned to it by anyone; whatever it responds to emerged from training because responding to that thing reduced the loss.

3.1.2 Why the activation function is not optional

Suppose we remove the activation and simply stack weighted sums. Layer one computes some combination of the inputs; layer two computes a combination of those combinations. It feels as though the network is getting more powerful.

It is not. A weighted sum of weighted sums is itself just a weighted sum. Stack a hundred layers without activations and you have, mathematically, exactly one layer — a single straight line, unable to solve XOR.

KEY TAKEAWAYS

The single most important sentence about neural networks

Non-linearity is what makes depth mean anything. Without an activation function, a deep network collapses to a shallow one. The activation is the small piece of machinery that lets each layer add genuinely new representational power rather than merely re-weighting what came before.

3.2 Activation Functions

Five you should know, and one you must know because it appears at the output of every language model.

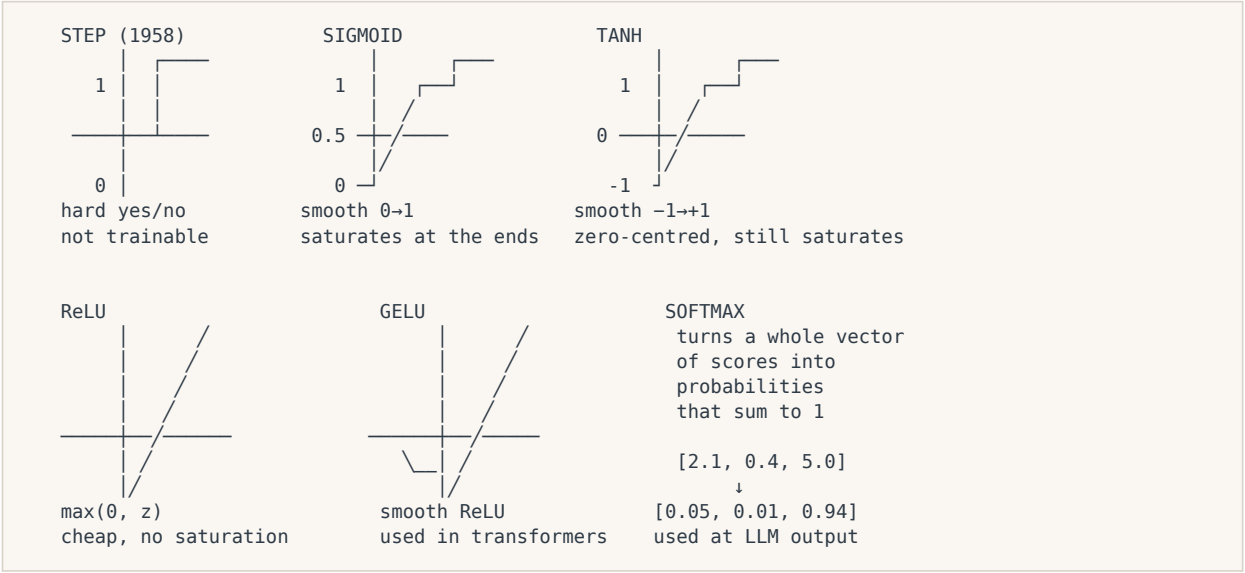


Figure 3.2 — The shapes of the common activation functions.

Activation	Formula (informally)	Where you meet it	Why it fell out of, or into, favour
Step	1 if `z > 0`, else 0	The 1958 perceptron	Has no usable slope, so gradient descent cannot see which way to move. Unusable for training.
Sigmoid	Squashes to the range 0–1	Output of binary classifiers	Saturates: for large positive or negative `z` the slope is almost zero, so gradients vanish and learning stalls.
Tanh	Squashes to the range −1 to +1	Older recurrent networks	Zero-centred, which helps, but still saturates.
ReLU	`max(0, z)`	The default in most deep networks since 2012	Cheap, no saturation for positive values, trains fast. Can "die" — a neuron stuck at zero output produces no gradient and never recovers.
GELU	A smoothed ReLU	Inside transformer feed-forward layers	Smooth near zero, which empirically trains better on language tasks. This is the one inside Claude.
Softmax	Exponentiate and normalise a whole vector	The final layer of every language model	Converts raw scores over the vocabulary into a probability distribution that sums to 1.

KEY TAKEAWAYS

Softmax is where Chapter 1 and Chapter 2 meet

In Figure 1.2 the model produced "a probability distribution over every token in the vocabulary". **Softmax is the function that produces it.**

The network's final layer emits one raw score per token — roughly 100,000 numbers, unbounded, meaningless on their own. Softmax exponentiates them and divides by the total, yielding numbers between 0 and 1 that sum to 1. Sampling (step 5 of Figure 1.2) then draws from that distribution.

This also explains **temperature**. Temperature divides the scores before softmax. A low temperature exaggerates the differences, concentrating probability on the top few tokens and making output more predictable. A high temperature flattens them, spreading probability and making output more varied. It is one division, applied to a vector, immediately before this function.

3.3 Layers, Networks and Parameter Counting

Connect neurons in parallel to form a **layer**, then stack layers. The result is a **multi-layer perceptron (MLP)**, also called a feed-forward or fully-connected network.

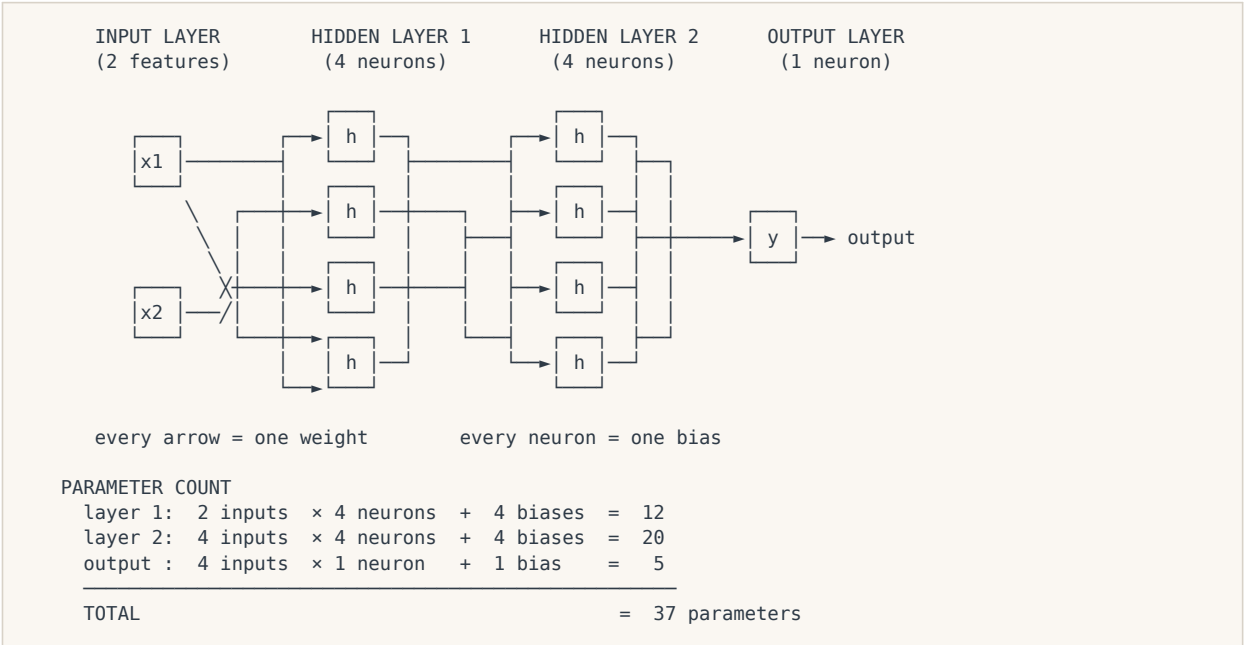


Figure 3.3 — A multi-layer perceptron with two hidden layers, solving XOR-like problems.

The counting rule is worth internalising because it is how you reason about model size:

```
parameters in a fully-connected layer  =  (inputs × outputs) + outputs
```

Two consequences follow immediately. Parameter count grows with the *product* of adjacent layer sizes, so widening a network is expensive. And a model described as having "70 billion parameters" is describing the sum of that arithmetic across all its layers — a statement about capacity and memory footprint, not about quality.

DEFINITION

Depth, width, hidden layer

Depth is the number of layers. **Width** is the number of neurons in a layer. A **hidden layer** is any layer that is neither the input nor the output — "hidden" only in the sense that its values are never directly observed or specified by you. **Deep learning** is simply learning with several hidden layers.

3.4 Why Depth Helps: The Feature Hierarchy

Depth is not just more capacity. It buys something qualitatively different: each layer learns to detect features built from the features detected by the layer below.

The clearest demonstration came from image networks, where the learned features can be visualised directly.

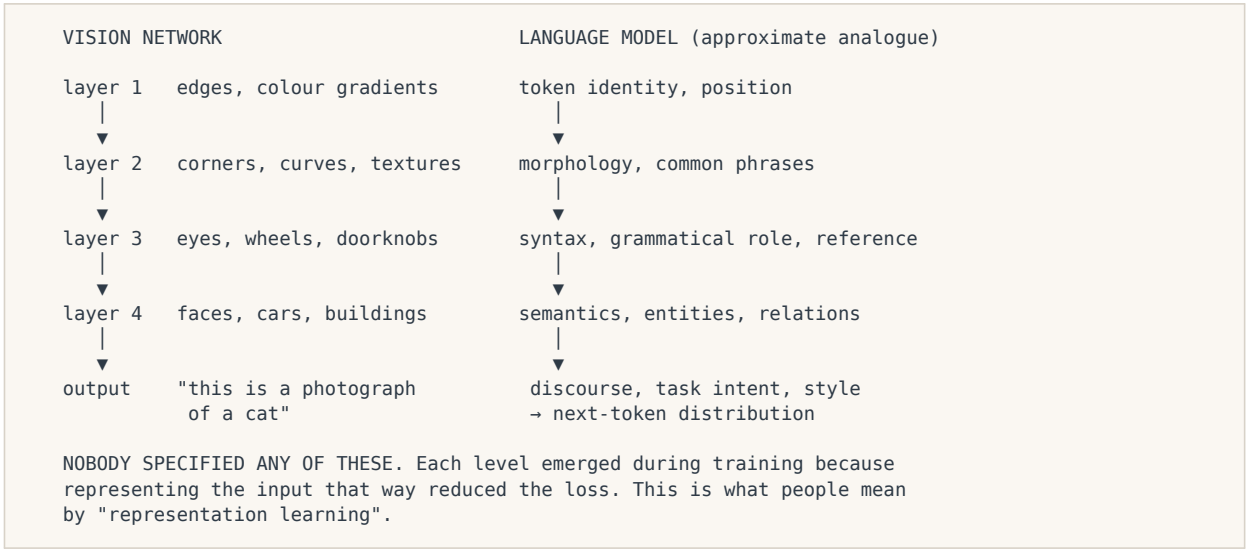


Figure 3.4 — The hierarchy of learned features, for vision and for language.

This is the answer to a question beginners reasonably ask: why not use one enormous wide layer instead of many narrow ones? Because a single layer must map raw input straight to the answer in one step. Depth allows the problem to be decomposed — solve an easier problem, then use its solution to solve the next. It is the same reason software is built from functions rather than one long procedure.

WARNING

Depth is not free

Deeper networks are harder to train. Gradients travelling backwards through many layers tend to shrink towards nothing (**vanishing gradients**) or grow explosively (**exploding gradients**). Modern architectures are largely a collection of fixes for this problem: better activations, normalisation layers, careful initialisation, and residual connections. Section 3.7 covers each. The transformer is, among other things, a design that made very deep networks trainable.

3.5 Backpropagation: Assigning Blame Backwards

Now the central mechanism. The question, restated: a network makes a prediction, we compute the loss, and we need to know — for **every one of possibly billions of parameters** — which direction to nudge it and how much. Doing this naively, by trying each parameter and seeing what happens, would require billions of forward passes per update. It is hopelessly infeasible.

Backpropagation computes all of them in a single backward sweep costing roughly the same as one forward pass.

The idea is easiest to see as a chain of responsibility.

VISUAL ANALOGY

The blame that flows down the org chart

A product ships late. The chief executive knows only the total delay: six weeks. Nobody knows which individual is responsible.

So the delay is apportioned **backwards, one management layer at a time**. The CEO tells three directors how much each contributed, based on how much each director's area feeds into the launch. Each director then apportions their share among their managers, weighted by how much each manager's work fed into theirs. Each manager apportions their share among engineers.

By the time this reaches the bottom, every individual has a number: *how much of the total delay was mine, and in which direction should I change*. And crucially, nobody had to simulate the entire company with each person's behaviour altered one at a time. Each layer only needed to know its own local contribution and the blame handed down from above.

That is backpropagation exactly. The "blame handed down" is the gradient. The "how much your work fed into mine" is the weight on the connection.

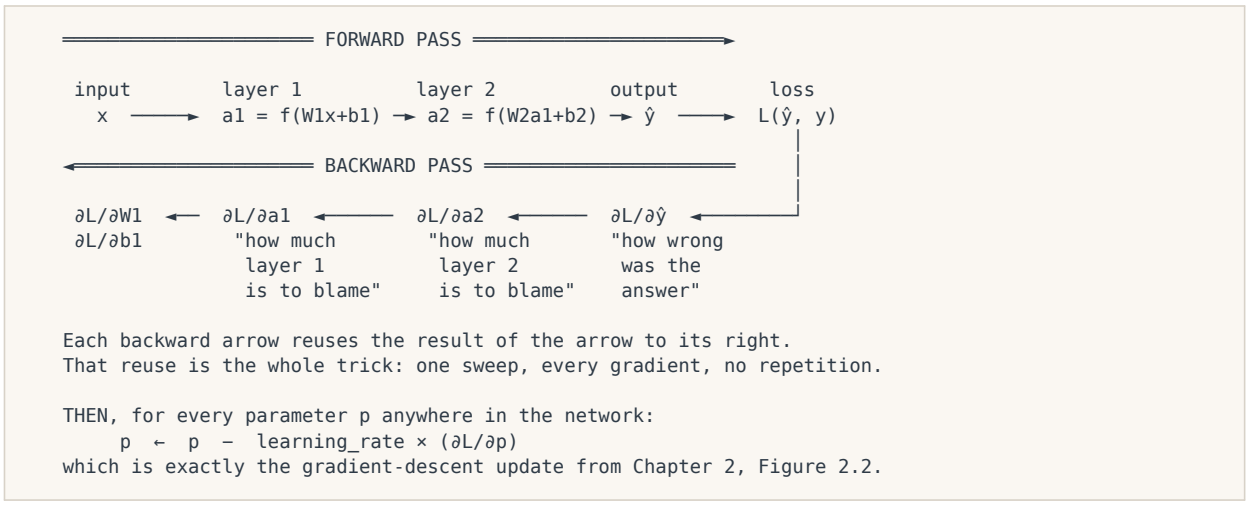


Figure 3.5 — One training step: forward for predictions, backward for blame.

NOTE

The one line of calculus, again stated and set aside

The multiplication of local sensitivities along a path is the **chain rule** of differential calculus. `∂L/∂W1` reads "the rate at which the loss changes as `W1` changes".

You will never compute one by hand. Every framework does it automatically — this is what **automatic differentiation** means, and it is why frameworks like PyTorch exist. The intuition in the analogy above is complete and correct for every purpose in this book, including the exam.

COMMON MISTAKE

"Backpropagation is how the model thinks"

It is not. Backpropagation happens **only during training**, and only on Anthropic's hardware, before the model was released.

When you call Claude, there is a forward pass and nothing else. No gradients are computed, no blame is assigned, no parameter changes. This is the mechanism behind the statelessness described in Chapter 1 and reinforced in Chapter 2, and it is worth being able to state precisely: *inference is forward-only*.

3.6 The Training Machinery You Will Meet Inside a Transformer

Plain gradient descent as described in Chapter 2 works, slowly and fragilely. Six refinements turned deep learning from a research curiosity into an industrial technique. You need to recognise all six by name, because they appear in every model card, every architecture diagram and every conversation with an ML engineer.

Technique	The problem it solves	What it does
Stochastic gradient descent (SGD)	Computing the gradient over millions of examples per step is far too slow	Estimate the gradient from a small random batch. Noisier per step, vastly faster overall, and the noise itself helps escape poor solutions.
Momentum	Descent zig-zags across narrow valleys and crawls along their floor	Accumulate a running average of recent gradients, like a ball rolling downhill gaining speed in a consistent direction.
Adam	One learning rate cannot suit every parameter	Maintain a per-parameter adaptive step size from the recent history of that parameter's gradients. The default optimiser for transformers.
Learning-rate schedule	Big steps are right early and wrong late	Warm up from near zero, then decay over training. Every large model is trained with a schedule; none uses a constant rate.
Normalisation (batch / layer norm)	Values drift to extreme scales as they pass through layers, destabilising training	Rescale activations to a consistent range at each layer. **Layer normalisation** is the variant used in transformers.
Dropout	Overfitting: the network memorises rather than generalises	During training, randomly switch off a fraction of neurons each step, forcing redundant representations. Disabled at inference.

Technique	The problem it solves	What it does
Residual (skip) connections	Gradients vanish through very deep stacks	Add each layer's input to its output, giving the gradient a direct path backwards. This is the single change that made networks hundreds of layers deep trainable, and it is present in every transformer block.

CERTIFICATION NOTE

Why an architect is asked to know these

You will not tune any of them. But the exam and real design conversations both turn on the *consequences*:

- **Dropout is off at inference.** A model behaves differently in training and serving mode. This is the general form of a class of production bugs.
- **Residual connections and layer norm are why transformers can be deep**, and depth is why they are capable. Understanding this makes "why is the big model better" a mechanical question rather than a mystery.
- **Training is fragile and expensive; inference is stable and cheap.** This asymmetry is the reason the industry converged on a small number of pre-trained models accessed by API rather than everyone training their own — and therefore the reason your job is architecture rather than training.

3.7 Matrices, and Why This Runs on GPUs

One more piece of mechanical understanding, because it explains the entire economics of the field.

A layer computes, for every neuron, a weighted sum of every input. Written out neuron by neuron this looks like a great many separate multiplications and additions. Written compactly, it is a single operation:

```
a  =  f( W · x  +  b )
W   a matrix of weights   (outputs × inputs)
x   a vector of inputs
b   a vector of biases
f   the activation, applied to every element independently
```

`W · x` is a **matrix multiplication**, and it has a property that changed everything: every entry of the result can be computed **independently of every other entry**, at the same time.

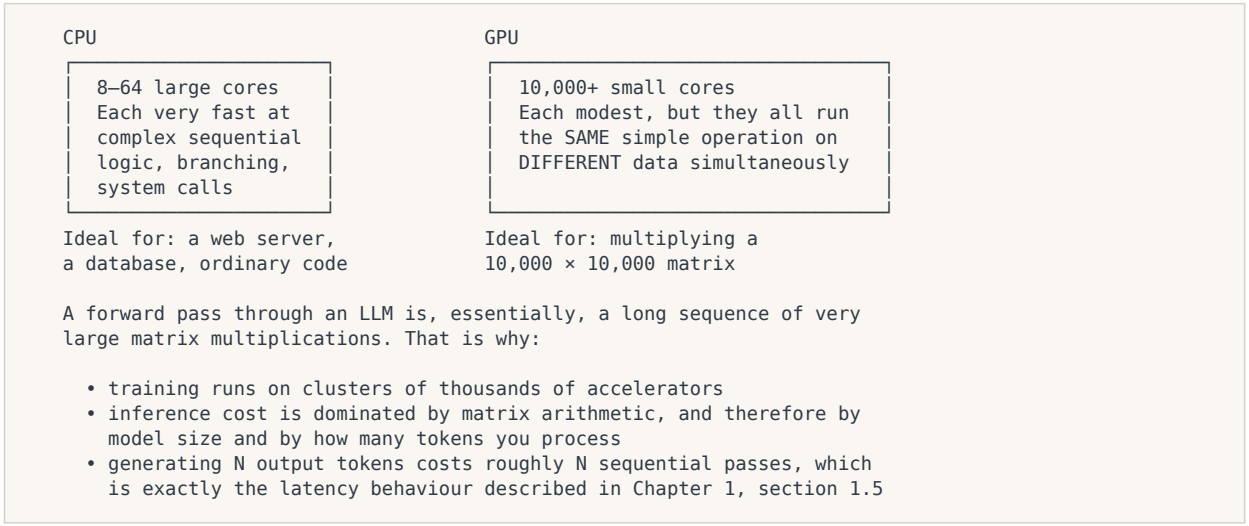


Figure 3.6 — Why the hardware looks the way it does.

KEY TAKEAWAYS

The chain from mathematics to your invoice

Layer = matrix multiplication → matrix multiplication parallelises → GPUs are built for parallel arithmetic → bigger models mean bigger matrices mean more arithmetic → **you are billed per token because tokens are the unit of arithmetic performed on your behalf**.

Every cost and latency fact in Parts 5, 10 and 12 traces back to this paragraph.

3.8 Where This Sits in a Claude-Powered System

Before the code, place the chapter in the architecture. You will never build the network inside Claude. What you gain from this chapter is the ability to reason correctly about its behaviour — and, occasionally, to build the small networks that surround it.

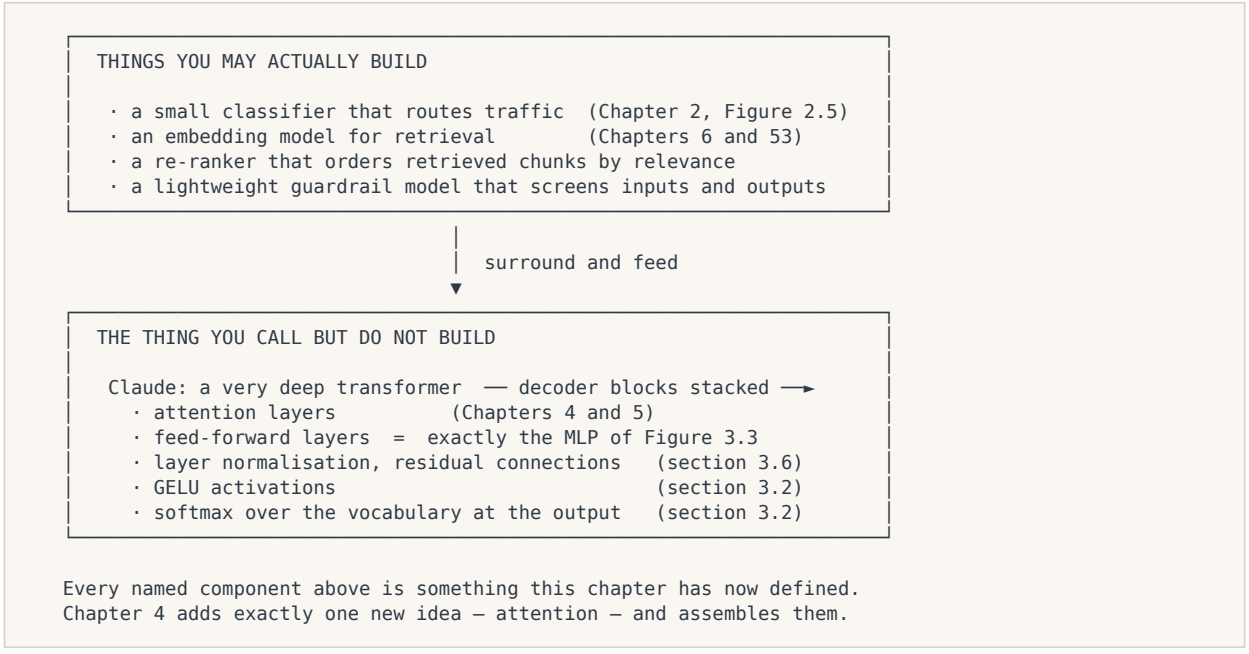


Figure 3.7 — What a chapter on neural networks buys an architect.

3.9 Code Walkthrough: A Network From Scratch, With Your Own Backpropagation

We will solve XOR — the problem that broke the field in 1969 — with a two-layer network, writing the backward pass by hand. Everything here runs; nothing is pseudocode.

```
import numpy as np
rng = np.random.default_rng(0)
# The four XOR cases. No straight line separates the outputs.
X = np.array([[0., 0.], [0., 1.], [1., 0.], [1., 1.]])
Y = np.array([[0.],     [1.],     [1.],     [0.]])
def sigmoid(z):
    return 1.0 / (1.0 + np.exp(-z))
H = 4                                       # width of the hidden layer
W1 = rng.normal(0, 1.0, (2, H))             # 2 inputs  -> 4 hidden
b1 = np.zeros((1, H))
W2 = rng.normal(0, 1.0, (H, 1))             # 4 hidden  -> 1 output
b2 = np.zeros((1, 1))
lr = 0.5
for epoch in range(20001):
    # ---------------------------------------------------- FORWARD PASS
    Z1 = X @ W1 + b1          # weighted sums, hidden layer
    A1 = np.maximum(0, Z1)    # ReLU activation
    Z2 = A1 @ W2 + b2         # weighted sums, output layer
    A2 = sigmoid(Z2)          # probability
    # ---------------------------------------------------- LOSS
    eps = 1e-9
    loss = -np.mean(Y * np.log(A2 + eps) + (1 - Y) * np.log(1 - A2 + eps))
    # ---------------------------------------------------- BACKWARD PASS
    dZ2 = (A2 - Y) / len(X)          # blame at the output
    dW2 = A1.T @ dZ2                 # ... attributed to W2
    db2 = dZ2.sum(axis=0, keepdims=True)
    dA1 = dZ2 @ W2.T                 # blame passed BACK to the hidden layer
    dZ1 = dA1 * (Z1 > 0)             # ReLU passes blame only where it fired
    dW1 = X.T @ dZ1
    db1 = dZ1.sum(axis=0, keepdims=True)
    # ---------------------------------------------------- UPDATE
    W2 -= lr * dW2
    b2 -= lr * db2
    W1 -= lr * dW1
    b1 -= lr * db1
    if epoch % 5000 == 0:
        print(f"epoch {epoch:6d}   loss {loss:.5f}")
print("\npredictions:", np.round(A2.ravel(), 3))
print("targets    :", Y.ravel())
```

Running it produces:

```
epoch      0   loss 0.83831
epoch   5000   loss 0.00072
epoch  10000   loss 0.00034
epoch  15000   loss 0.00022
epoch  20000   loss 0.00016
predictions: [0. 1. 1. 0.]
targets    : [0. 1. 1. 0.]
```

The four lines that matter, explained.

`dZ2 = (A2 - Y) / len(X)` — the blame at the output. As in Chapter 2, for a sigmoid output with cross-entropy loss this reduces to simply *how far off you were*, averaged over the batch. Nothing more complicated is happening here.

`dW2 = A1.T @ dZ2` — attributing that blame to the weights. A weight is more responsible for the error the larger the activation it multiplied. A connection carrying a near-zero signal cannot have caused much of the mistake, and correspondingly receives a near-zero gradient.

`dA1 = dZ2 @ W2.T` — **this single line is backpropagation.** The blame is pushed backwards through the weight matrix, distributing it to the hidden neurons in proportion to how strongly each one influenced the output. This is the director apportioning delay among their managers.

`dZ1 = dA1 * (Z1 > 0)` — the ReLU gate on the way back. A neuron whose pre-activation was negative output zero and therefore contributed nothing, so it receives no blame. This is also the mechanism behind the **dying ReLU** problem: a neuron stuck permanently negative receives zero gradient forever and can never recover.

> NOTE
>
> **What a framework changes, and what it does not**
>
> The same network in PyTorch is about a dozen lines, and the backward pass disappears entirely:
>
> ```
> loss.backward()      # PyTorch computes every gradient automatically
> optimizer.step()     # and applies the update
> ```
>
> Those two lines do exactly what the twelve lines above do. Automatic differentiation records the operations of the forward pass and replays them backwards. You will use frameworks in practice — but having written the backward pass once, you will never again mistake it for magic, and you will be able to reason about vanishing gradients, dying neurons and initialisation from mechanism rather than folklore.

3.10 Mini Project: A Neural Language Model in 60 Lines

Goal. Build a character-level neural language model: it looks at the previous four characters and predicts the next one. Train it, then generate text from it.

This is not an analogy for what Claude does. It is a miniature of it, differing in scale, in architecture (transformer rather than MLP), and in unit (tokens rather than characters) — but identical in objective, in loss function, and in the way text is generated one unit at a time.

Architecture.

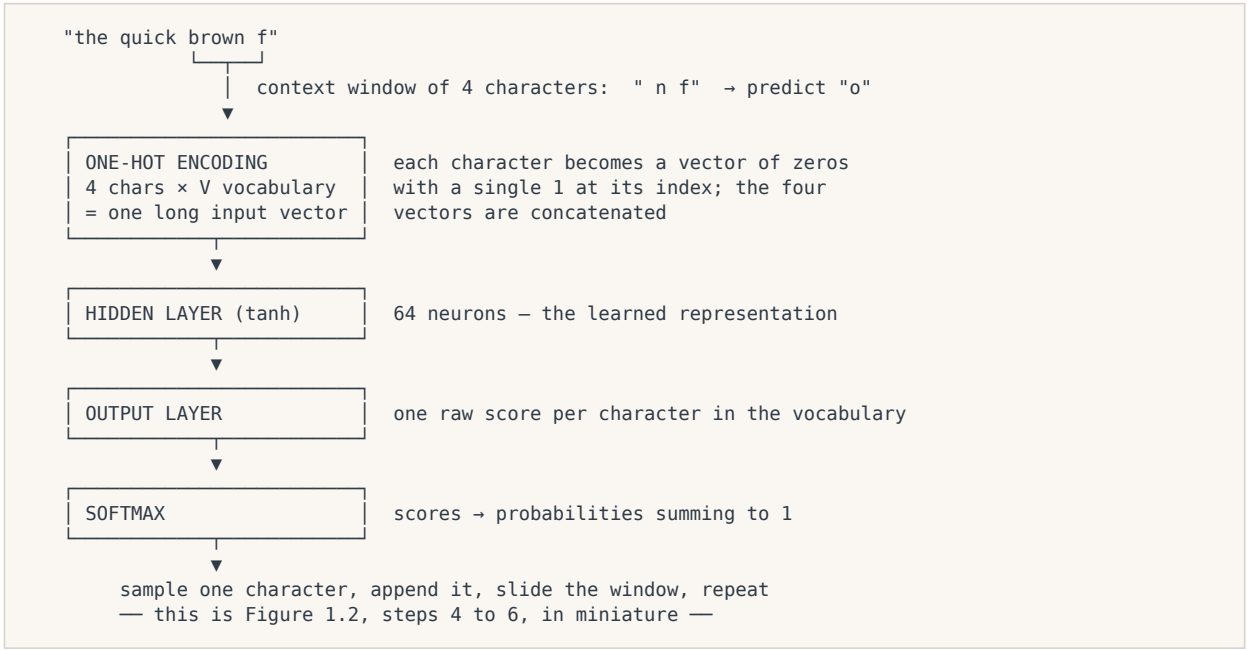


Figure 3.8 — A character-level neural language model.

Folder structure.

```
tiny-lm/
├── README.md
├── requirements.txt          # numpy
├── tinylm/
│   ├── __init__.py
│   ├── data.py               # corpus, vocabulary, encoding
│   ├── model.py              # forward, backward, train
│   └── generate.py           # sampling with temperature
└── tests/
    └── test_tinylm.py
```

Complete code.

```
# tinylm/model.py
import numpy as np
rng = np.random.default_rng(1)
TEXT = ("the quick brown fox jumps over the lazy dog. " * 40 +
        "pack my box with five dozen liquor jugs. " * 40)
chars = sorted(set(TEXT))
V = len(chars)
stoi = {c: i for i, c in enumerate(chars)}
itos = {i: c for c, i in stoi.items()}
CTX, H = 4, 64          # context length, hidden width
def encode_ctx(seq):
    """CTX characters -> one concatenated one-hot vector of length CTX*V."""
    v = np.zeros(CTX * V)
    for k, ch in enumerate(seq):
        v[k * V + stoi[ch]] = 1.0
    return v
# Build every (context, next character) pair in the corpus.
# This is self-supervised learning from Chapter 2, section 2.2.4.
Xs, Ys = [], []
for i in range(len(TEXT) - CTX):
    Xs.append(encode_ctx(TEXT[i:i + CTX]))
    Ys.append(stoi[TEXT[i + CTX]])
```

```
X, Y = np.array(Xs), np.array(Ys)
W1 = rng.normal(0, 0.05, (CTX * V, H)); b1 = np.zeros(H)
W2 = rng.normal(0, 0.05, (H, V));       b2 = np.zeros(V)
def softmax(z):
    z = z - z.max(axis=1, keepdims=True)      # numerical stability
    e = np.exp(z)
    return e / e.sum(axis=1, keepdims=True)
def train(epochs=60, lr=0.5, batch=128):
    global W1, b1, W2, b2
    n = len(X)
    for epoch in range(epochs):
        perm = rng.permutation(n)             # stochastic gradient descent
        total = 0.0
        for s in range(0, n, batch):
            idx = perm[s:s + batch]
            xb, yb = X[idx], Y[idx]
            # forward
            A1 = np.tanh(xb @ W1 + b1)
            P = softmax(A1 @ W2 + b2)
            total += -np.log(P[np.arange(len(yb)), yb] + 1e-9).sum()
            # backward
            dZ2 = P.copy()
            dZ2[np.arange(len(yb)), yb] -= 1
            dZ2 /= len(yb)
            dW2 = A1.T @ dZ2
            db2 = dZ2.sum(0)
            dZ1 = (dZ2 @ W2.T) * (1 - A1 ** 2)     # tanh derivative
            dW1 = xb.T @ dZ1
            db1 = dZ1.sum(0)
            W2 -= lr * dW2; b2 -= lr * db2
            W1 -= lr * dW1; b1 -= lr * db1
        if epoch % 20 == 0:
            print(f"epoch {epoch:3d}  loss {total / n:.4f}")
```

```
# tinylm/generate.py
import numpy as np
from .model import W1, b1, W2, b2, softmax, encode_ctx, itos, CTX, rng
def generate(seed, n_chars=80, temperature=0.8):
    out = seed
    for _ in range(n_chars):
        ctx = out[-CTX:].rjust(CTX)               # pad with spaces if short
        A1 = np.tanh(encode_ctx(ctx)[None, :] @ W1 + b1)
        logits = (A1 @ W2 + b2) / temperature     # temperature, section 3.2
        p = softmax(logits)[0]
        out += itos[rng.choice(len(p), p=p)]      # sampling, Figure 1.2 step 5
    return out
```

What it produces. After sixty epochs the loss falls from about 3.03 — which is roughly the loss of guessing uniformly at random from the vocabulary — to about 0.03, and generation from the seed `"the "` yields:

```
the lazy dog. the quick brown fox jumps over the lazy dog. the quick brown fox
```

Testing.

```
# tests/test_tinylm.py
import numpy as np
from tinylm.model import softmax, encode_ctx, CTX, V
```

```
def test_softmax_is_a_distribution():
    p = softmax(np.array([[2.1, 0.4, 5.0]]))
    assert np.isclose(p.sum(), 1.0)
    assert p.argmax() == 2
def test_encoding_shape_and_sparsity():
    v = encode_ctx("abcd")
    assert v.shape == (CTX * V,)
    assert v.sum() == CTX          # exactly one 1 per context position
```

Evaluation. Hold back the last 10% of the corpus, never train on it, and report loss on it separately. If training loss falls while held-out loss rises, you have reproduced Figure 2.4 in a language model — which is exactly what happens when a real model is trained too long on too little data.

KEY TAKEAWAYS

What this project actually demonstrates

Six things, and each one recurs for the rest of the book:

1. **The training data was created for free** by sliding a window over raw text — self-supervision, Chapter 2.
2. **The loss is cross-entropy over the next unit**, which is the LLM objective.
3. **The output is a probability distribution**, produced by softmax.
4. **Text is generated one unit at a time**, each appended and fed back — Figure 1.2, steps 4 to 6.
5. **Temperature is a division before softmax.** Set it to 0.3 and the output becomes repetitive; set it to 1.5 and it degenerates into noise. You can watch this happen.
6. **The context window is a hard architectural limit.** This model sees exactly four characters. Ask it to close a bracket opened ten characters ago and it cannot — the information is not in its input. Claude's window is a million tokens rather than four characters, but *the limit is the same kind of limit*, and everything in Part 10 follows from that.

3.11 Hands-on Labs

Lab 3.1 — Prove the 1969 result. Delete the hidden layer from the XOR network so that inputs connect directly to the output. Train for 20,000 epochs. *Success criterion:* loss plateaus near 0.69 and predictions sit near 0.5 for all four cases. Explain, geometrically, why.

Lab 3.2 — Remove the non-linearity. Restore the hidden layer but replace `np.maximum(0, Z1)` with `Z1` — no activation. Train again. *Success criterion:* it fails as completely as Lab 3.1, and you can explain why two linear layers are one linear layer.

Lab 3.3 — Sweep the width. Run the XOR network with `H` set to 1, 2, 4, 16 and 64. Record final loss and epochs to converge. *Success criterion:* a table, plus a statement about the minimum width that solves XOR and why.

Lab 3.4 — Kill some neurons. Initialise `b1` to a large negative value, for example `-5.0`, so every ReLU starts switched off. Train. *Success criterion:* you observe the dying-ReLU failure and can trace it to the `(Z1 > 0)` gate in the backward pass.

Lab 3.5 — Sweep the temperature. In the mini project, generate with temperature 0.1, 0.5, 1.0 and 2.0 from the same seed. *Success criterion:* four samples, plus a one-paragraph account of the trade-off,

connected to the sampling step of Figure 1.2.

3.12 Lab Solutions

Solution 3.1. Without a hidden layer the model is a single neuron: a weighted sum through a sigmoid, which partitions the input space with a straight line. The XOR targets place `(0,0)` and `(1,1)` in one class and `(0,1)` and `(1,0)` in the other; those two pairs sit on opposing diagonals and no line separates them. The best achievable is to predict 0.5 everywhere, whose cross-entropy is `-ln(0.5) ≈ 0.693`. Seeing that exact number appear is a satisfying confirmation that the model is not broken — it is at its theoretical optimum, and its optimum is useless.

Solution 3.2. With `A1 = Z1`, the network computes `(X W1 + b1) W2 + b2`, which expands to `X (W1 W2) + (b1 W2 + b2)`. The products in brackets are just some other matrix and some other vector — that is, a single linear layer. Depth without non-linearity is decorative. This is section 3.1.2 demonstrated rather than asserted.

Solution 3.3. Two hidden neurons are sufficient in principle: one can learn "at least one input is on", the other "both inputs are on", and the output layer subtracts. In practice `H=2` often fails from an unlucky initialisation, because there is no redundancy to recover from a dead neuron. `H=4` succeeds reliably. Beyond about 16, additional width buys nothing on a four-example problem and simply adds parameters. This is the capacity-versus-need trade-off from Chapter 2 at the smallest possible scale.

Solution 3.4. With every pre-activation negative, all ReLUs output zero, so the output layer receives nothing and, on the backward pass, `(Z1 > 0)` is false everywhere and `dZ1` is zero throughout. No hidden parameter can ever change. The network is permanently frozen at initialisation. Fixes: sensible initialisation, a leaky variant of ReLU that passes a small gradient for negative inputs, or normalisation to keep pre-activations centred. This is why section 3.6 exists.

Solution 3.5. At 0.1 the output is near-deterministic and quickly falls into a repeating loop, because the single most likely character is chosen almost every time. At 1.0 it follows the learned distribution faithfully. At 2.0 the distribution is flattened towards uniform and output degenerates into noise. The trade-off is exactly the one you make in production: low temperature for extraction and classification where you want the most probable answer, higher for creative generation where variety is the point.

3.13 Best Practices

BEST PRACTICE

Seven habits, even though you will not train the big models

1. **Always establish what random guessing scores.** For a 40-character vocabulary that is a cross-entropy of about `ln(40) ≈ 3.7`. Without that reference, "loss is 2.9" means nothing.
2. **Watch training and held-out loss together.** One curve alone cannot distinguish learning from memorising.
3. **Overfit a tiny sample deliberately first.** If your model cannot drive the loss to near zero on ten examples, there is a bug in the code, not a shortage of data. This is the fastest debugging technique in machine learning.
4. **Check tensor shapes before checking your reasoning.** The overwhelming majority of neural-network bugs are a transposed matrix or a missing axis.
5. **Fix the random seed while developing.** Non-determinism plus a real bug is an unpleasant combination.
6. **Log the loss every epoch and look at the curve, not the final number.** Divergence, plateaus and instability are all visible in the shape and invisible in the endpoint.
7. **Prefer the smallest model that meets the requirement.** The same instinct that chooses a classifier over an LLM in Chapter 2 chooses four hidden units over four hundred here.

3.14 Common Mistakes

COMMON MISTAKE

Believing the model reasons by backpropagating

Symptom: a design document describing the agent as "back-propagating from the goal". **Cause:** conflating training with inference. **Fix:** inference is forward-only; nothing changes during a request. **Detection:** any claim about run-time learning without a training pipeline in the diagram.

COMMON MISTAKE

Stacking layers without activations

Symptom: a deep network that performs no better than a linear model. **Cause:** the network is mathematically linear. **Fix:** a non-linearity after every hidden layer. **Detection:** performance identical to logistic regression is the tell.

COMMON MISTAKE

Treating parameter count as a quality score

Symptom: "our model has more parameters, so it is better." **Cause:** confusing capacity with capability. **Fix:** capability depends on data, training compute, architecture and alignment as much as on size. A well-trained smaller model routinely outperforms a poorly trained larger one — which is precisely why the Haiku tier exists and is often the right choice.

COMMON MISTAKE

Ignoring the train/inference behaviour split

Symptom: metrics that differ between evaluation and production for no apparent reason. **Cause:** dropout and normalisation behave differently in the two modes. **Fix:** set the model to evaluation mode explicitly. **Detection:** a persistent, unexplained gap between offline and online scores.

COMMON MISTAKE

Assuming a bigger context window is an architectural change

Symptom: expecting a 1M-token window to remove the need for retrieval. **Cause:** treating the window as memory rather than as input. **Fix:** the window is the size of the input vector, not a store. Bigger inputs mean more arithmetic, more cost, more latency and more dilution of attention. Chapter 55 quantifies this.

3.15 Performance Considerations

Parameters determine memory; tokens determine work. A model's parameter count fixes how much memory the weights occupy. The arithmetic performed on a request scales with the number of tokens processed. These are separate axes, and confusing them leads to bad capacity planning.

Generation is inherently sequential. Each output token requires a full pass through the network before the next can begin. This is a property of the algorithm, not an implementation limitation, and it is why output length dominates total latency. Techniques exist to mitigate it, but none removes the sequential dependency.

Numeric precision is a cost lever. Models are commonly served at reduced precision — fewer bits per weight — which cuts memory and increases throughput at a small and usually acceptable cost in quality. You do not control this on a hosted API, but it explains why serving costs fall over time for the same model.

Batching helps throughput, not latency. Processing many requests together uses the hardware far more efficiently, which is why batch endpoints are cheaper. It does nothing for the latency of any individual request, and can worsen it. Match the mechanism to the requirement: interactive work wants streaming, bulk work wants batching.

3.16 Security Considerations

Adversarial examples exist. Inputs can be constructed that a network misclassifies with high confidence while looking unremarkable to a person. The language-model equivalent is the crafted prompt that reliably elicits behaviour the model would otherwise decline. The general lesson is that a model's decision boundary has structure nobody designed and nobody fully understands, and an attacker with query access can search it.

Models can be extracted. Given enough queries, an adversary can train a substitute model that imitates yours. If a model represents commercial value, rate limiting and query monitoring are security controls, not merely cost controls.

Backdoors survive training. A model trained on poisoned data can behave correctly everywhere except on a specific trigger. This is undetectable by ordinary evaluation, because ordinary evaluation does not contain the trigger. Provenance of weights and of training data is a genuine supply-chain concern.

Never treat model output as trusted input. A network produces a plausible continuation of its input. Nothing in its construction constrains it to produce something safe to execute, insert into a query, or render as markup. Validation belongs between the model and every consumer of its output. This is the Chapter 1 principle restated with the mechanism now visible.

3.17 Real Production Examples

Embedding models for retrieval. A network maps text to a vector such that similar meanings land near one another. This is the foundation of the retrieval systems in Part 10 — and it is a neural network of exactly the kind built in this chapter, trained with a different objective.

Re-rankers. After a retrieval step returns fifty candidate passages, a small network scores each against the query and reorders them. Cheap, fast, and often a larger quality improvement than any change to the prompt.

Guardrail classifiers. Small networks screening inputs for injection attempts and outputs for policy violations. They run in milliseconds alongside a much slower generation call, so they are effectively free in latency terms.

Speculative decoding. A small fast model proposes several tokens; the large model verifies them in one pass. Where the proposals are accepted, several tokens are produced for the cost of roughly one. This is a production technique that only makes sense once you understand that generation is sequential and that verification can be parallel.

3.18 Certification Notes

Neural-network internals are not directly examined. Four ideas from this chapter, however, underpin scenarios across every domain.

Concept	Where it matters on the exam
Inference is forward-only; parameters are frozen	Domains 1 and 5: statelessness, subagent context isolation, why memory must be built
Softmax and temperature	Domain 4: output determinism, and why Opus 5 and Sonnet 5 reject non-default sampling parameters
The context window is the input, not a memory	Domain 5: the whole of context management
Sequential generation	Domains 1 and 5: latency budgeting, streaming, why concise outputs are faster

CERTIFICATION NOTE

Exam tips carried forward

- Anything asserting run-time learning is wrong. There is no backward pass at inference.
- Anything asserting that a larger context window removes the need for retrieval or compaction is wrong. Larger input means more cost, more latency and more dilution.
- Options claiming that a bigger model is always the right answer are wrong more often than not. The exam consistently rewards the smallest sufficient mechanism.
- Latency questions usually hinge on output length. If a scenario has a strict response-time budget, look for the option that shortens or streams the output, not the one that shortens the prompt.

3.19 Practice Questions

1. Why can a single neuron not solve XOR? (a) It has too few parameters. (b) It computes a linear boundary, and XOR is not linearly separable. (c) Its activation function is too slow. (d) XOR requires more training data.

2. What happens if you stack ten layers with no activation functions? (a) The network becomes ten times more expressive. (b) It is mathematically equivalent to a single linear layer. (c) Training becomes unstable but capability increases. (d) Gradients explode.

3. What does softmax do? (a) Selects the largest value in a vector. (b) Converts a vector of raw scores into a probability distribution summing to 1. (c) Normalises activations to zero mean. (d) Randomly drops neurons during training.

4. Temperature, in sampling, is applied: (a) After softmax, to the probabilities. (b) Before softmax, dividing the raw scores. (c) During backpropagation. (d) To the loss function.

5. A fully-connected layer with 128 inputs and 64 outputs has how many parameters? (a) 8,192 (b) 8,256 (c) 192 (d) 64

6. Backpropagation is best described as: (a) The model reasoning backwards from a goal. (b) An efficient single backward sweep that computes the gradient for every parameter. (c) Re-running the forward pass once per parameter. (d) A method of compressing model weights.

7. When does backpropagation occur in a production Claude application? (a) On every request. (b) When the user corrects the model. (c) Never; it happened during training only. (d) During streaming.

8. Residual connections primarily address: (a) Overfitting. (b) Gradients vanishing as they travel back through many layers. (c) Tokenization inefficiency. (d) Memory usage at inference.

9. Dropout is: (a) Applied during training and disabled at inference. (b) Applied at inference to add variety. (c) A form of learning-rate schedule. (d) A normalisation technique.

10. The "dying ReLU" problem occurs because: (a) ReLU saturates for large positive inputs. (b) A neuron with a negative pre-activation outputs zero and receives zero gradient. (c) ReLU is computationally expensive. (d) ReLU cannot represent negative numbers in the output layer.

11. Why are GPUs suited to neural networks? (a) They have faster individual cores than CPUs. (b) Matrix multiplication parallelises, and GPUs run many identical operations at once. (c) They have more memory

than CPUs. (d) They handle branching logic better.

12. Which is the correct description of depth in a network? (a) The number of neurons per layer. (b) The number of layers. (c) The number of training epochs. (d) The size of the input vector.

13. A network drives training loss to near zero but performs poorly on held-out data. The best first response is: (a) Add more layers. (b) Reduce capacity, add regularisation, or gather more data. (c) Increase the learning rate. (d) Train for more epochs.

14. Which activation is used inside transformer feed-forward layers? (a) Step (b) Sigmoid (c) GELU (d) Softmax

15. In the character-level model of section 3.10, what limits how far back the model can see? (a) The hidden layer width. (b) The context length used to build the input vector. (c) The learning rate. (d) The vocabulary size.

16. Adam is: (a) A loss function. (b) An optimiser that adapts the step size per parameter. (c) A normalisation layer. (d) An activation function.

17. A model is described as having 70 billion parameters. This tells you: (a) That it is more accurate than a 7-billion-parameter model. (b) Its capacity and memory footprint, not its quality. (c) How many tokens it can process. (d) How many training examples it saw.

18. Which statement about generation latency is correct? (a) It is dominated by the number of output tokens, because generation is sequential. (b) It is dominated by the size of the context window. (c) It is constant regardless of output length. (d) It is determined by the number of tools defined.

19. Layer normalisation is used to: (a) Prevent overfitting by dropping neurons. (b) Keep activation scales consistent so deep networks train stably. (c) Convert scores into probabilities. (d) Reduce the number of parameters.

20. A colleague proposes that the agent will "learn from corrections and improve over the week". What is the correct architectural response? (a) Agree; models adapt continuously. (b) Explain that parameters are frozen and propose storage plus retrieval to persist corrections into future prompts. (c) Propose increasing the temperature. (d) Propose a larger context window as the sole fix.

3.20 Answers and Explanations

1 — (b). A single neuron partitions the input space with a hyperplane. XOR's classes lie on opposing diagonals and cannot be separated by one. More data or more parameters within a single neuron cannot change this.

2 — (b). Composition of linear maps is linear. This is why non-linearity is the enabling ingredient for depth.

3 — (b). Exponentiate and normalise, producing a valid distribution. (a) describes argmax, which is a different operation and discards the distribution.

4 — (b). Temperature divides the logits before softmax. Low temperature sharpens the distribution; high temperature flattens it.

5 — (b). `128 × 64 = 8,192` weights, plus 64 biases, giving 8,256. Forgetting biases is the standard error.

6 — (b). One backward sweep, every gradient, at roughly the cost of one forward pass. (a) is the anthropomorphic misreading the exam likes to punish; (c) describes the infeasible naive method.

7 — (c). Training only. This is the mechanical basis of statelessness.

8 — (b). Residual connections give the gradient a direct path backwards, which is what made very deep stacks trainable.

9 — (a). Training only. The train/inference behaviour split is a real source of production bugs.

10 — (b). Zero output means zero gradient through the ReLU gate, so the neuron can never recover.

11 — (b). Massive parallelism suited to matrix arithmetic. Individual GPU cores are slower than CPU cores, not faster.

12 — (b). Depth is layers; width is neurons per layer.

13 — (b). This is the overfitting signature from Chapter 2. Adding capacity or training longer makes it worse.

14 — (c). GELU. Softmax appears at the output over the vocabulary, not inside the feed-forward block.

15 — (b). The context length is a hard architectural limit — information outside it is simply not in the input. This generalises directly to the context window of a production model.

16 — (b). A per-parameter adaptive optimiser, and the default for transformer training.

17 — (b). Capacity and memory. Capability depends on data, compute, architecture and alignment as well.

18 — (a). Sequential generation makes output length the dominant term. Input length mostly affects time to first token.

19 — (b). Keeping activation scales stable across layers, which is what makes deep stacks trainable.

20 — (b). Frozen parameters plus explicit persistence. This exact scenario, in various disguises, appears throughout Domains 1 and 5.

3.21 Ten Practical Exercises

1. Extend the XOR network to three layers and verify it still converges. Note how many more epochs it needs and hypothesise why.
2. Implement leaky ReLU and repeat Lab 3.4. Confirm the network recovers.
3. Add momentum to the XOR trainer and measure the reduction in epochs to convergence.
4. Reimplement the XOR network in PyTorch and confirm it produces the same result with `loss.backward()`.
5. Instrument the mini project to report held-out loss each epoch and identify the epoch at which overfitting begins.
6. Increase `CTX` in the mini project from 4 to 8 and describe both the quality change and the parameter-count change.

7. Train the mini project on a corpus of your own — a source file, a poem, a log file — and characterise what it learns.
8. Implement greedy decoding, which always takes the highest-probability character, and explain why the output loops.
9. Compute by hand the parameter count of a network with layer sizes 784, 256, 128, 10. Verify with code.
10. Write a one-page explanation of backpropagation for a non-technical colleague using an analogy that is not the org chart.

3.22 Five Challenge Labs

Challenge 1 — A minimal autodiff engine. Implement a scalar `Value` class that records operations and supports `backward()`, then rebuild the XOR network on top of it. This is the single most instructive exercise in the chapter.

Challenge 2 — Gradient checking. For every parameter in the XOR network, estimate the gradient numerically by nudging the parameter slightly and observing the loss change, and compare with your analytical gradient. Any mismatch is a bug — this is how framework authors test their own code.

Challenge 3 — The initialisation study. Train the same network from ten different random initialisations at three different scales. Report the distribution of outcomes. Explain why initialisation scale matters and connect it to vanishing and exploding gradients.

Challenge 4 — Temperature and repetition. Extend the mini project to measure repetition rate — the fraction of generated substrings that already appear in the output — across a temperature sweep. Plot quality against temperature and identify the useful band.

Challenge 5 — The context-length experiment. Train the mini project at context lengths 1, 2, 4, 8 and 16. Plot held-out loss against context length, and against parameter count. You have now empirically derived, at small scale, the argument for large context windows — and the cost curve that accompanies them.

3.23 Interview Questions

1. Explain why a neural network needs non-linear activation functions.
2. Describe backpropagation without using the word "derivative".
3. What is the difference between a parameter and an activation?
4. Why did deep learning become practical in 2012 rather than 1986?
5. Explain softmax and temperature, and how they relate to the variability of LLM output.
6. What is the vanishing gradient problem, and name two things that mitigate it.
7. Why do neural networks run on GPUs rather than CPUs?
8. A model performs well in evaluation and poorly in production. Give four possible causes.
9. What does a model's parameter count tell you, and what does it not tell you?
10. Explain to a product manager why a shorter response is faster, in terms of how generation works.

3.24 Summary

A neuron is a weighted sum plus a bias, passed through a non-linear activation function — precisely the logistic-regression model of Chapter 2 with one component named. Connect neurons in parallel to form layers and stack the layers to form a network. Without the non-linearity, stacking achieves nothing: any number of linear layers collapses into one, which is why a single perceptron could not solve XOR and why the field stalled for a decade over it.

Depth matters because each layer learns features composed from the features of the layer below. Nobody specifies these; they emerge because representing the input that way reduces the loss. This is representation learning, and it is what separates deep learning from everything before it.

Backpropagation makes deep networks trainable by computing every parameter's gradient in one backward sweep, apportioning blame layer by layer in proportion to each connection's contribution. Around it sits a standard toolkit — stochastic gradient descent, momentum, Adam, learning-rate schedules, normalisation, dropout and residual connections — each of which fixes a specific failure mode, and all of which are present in the model you call as Claude.

Because a layer is a matrix multiplication and matrix multiplication parallelises, all of this runs on hardware built for parallel arithmetic. That fact propagates all the way to your invoice: tokens are the unit of arithmetic, which is why they are the unit of billing and the unit of latency.

Finally, the sixty-line character model in section 3.10 is not a metaphor for a large language model. It is one, in miniature: self-supervised training data, cross-entropy over the next unit, softmax at the output, sampling with temperature, generation one unit at a time, and a hard context limit. Chapter 4 replaces its architecture with a transformer, and Chapter 5 explains the one new idea — attention — that makes the replacement worth making.

KEY TAKEAWAYS

1. **A neuron is a weighted sum plus a bias plus a non-linearity.** Nothing more.
2. **Without activations, depth is worthless** — the network collapses to a single linear layer.
3. **Depth buys a hierarchy of learned features**, none of which anyone designed.
4. **Backpropagation assigns blame backwards** in one efficient sweep. It happens only during training.
5. **Inference is forward-only.** This is the mechanical basis of statelessness.
6. **Softmax turns scores into a distribution; temperature is a division applied just before it.**
7. **Residual connections, layer norm and Adam** are why very deep networks became trainable.
8. **A layer is a matrix multiplication**, which is why this runs on GPUs and why tokens cost money.
9. **Parameter count is capacity, not quality.** The smallest sufficient model is usually the right one.
10. **Generation is sequential**, so output length dominates latency — a fact you will design around for the rest of the book.

3.25 Glossary

Term	Definition
Activation	The output value of a neuron after its activation function is applied.
Activation function	The non-linearity applied to a neuron's weighted sum. Without it, depth is meaningless.
Adam	An optimiser that adapts the step size per parameter; the default for transformer training.
Automatic differentiation	Framework machinery that computes gradients by recording and replaying the forward pass.
Backpropagation	The algorithm that computes every parameter's gradient in one backward sweep.
Bias	A per-neuron constant added to the weighted sum, shifting its activation threshold.
Depth	The number of layers in a network.
Dropout	Randomly disabling neurons during training to reduce overfitting. Disabled at inference.
Dying ReLU	A neuron permanently outputting zero, receiving no gradient and unable to recover.
Exploding gradient	Gradients growing uncontrollably as they propagate backwards through many layers.
Feed-forward network	A network where information flows in one direction, input to output. Also multi-layer perceptron.
GELU	A smoothed variant of ReLU used in transformer feed-forward layers.
Hidden layer	Any layer that is neither input nor output.
Layer normalisation	Rescaling activations within a layer to keep their scale stable; standard in transformers.
Learning-rate schedule	A plan for varying the step size over training, typically warm-up then decay.
Logit	A raw, unnormalised score produced before softmax.
Momentum	Accumulating a running average of gradients to smooth and accelerate descent.
Multi-layer perceptron	A stack of fully-connected layers with non-linear activations.
Neuron	A unit computing a weighted sum plus bias, passed through an activation function.
ReLU	`max(0, z)`. The standard activation in most deep networks since 2012.
Representation learning	The emergence of useful internal features during training, without being specified.
Residual connection	Adding a layer's input to its output, giving gradients a direct path backwards.
Softmax	Converts a vector of scores into a probability distribution summing to one.
Stochastic gradient descent	Estimating the gradient from a random batch rather than the whole dataset.
Tanh	An activation squashing values to the range −1 to +1.
Temperature	A divisor applied to logits before softmax, controlling output variability.
Vanishing gradient	Gradients shrinking towards zero as they propagate backwards, stalling learning in early layers.

Term	Definition
Weight	A learned multiplier on one connection between units.
Width	The number of neurons in a layer.

3.26 Further Reading

Primary and canonical sources.

- Rumelhart, Hinton and Williams (1986), *Learning representations by back-propagating errors*. The paper that made deep networks trainable.
- Krizhevsky, Sutskever and Hinton (2012), *ImageNet Classification with Deep Convolutional Neural Networks*. The result that started the modern era.
- He et al. (2015), *Deep Residual Learning for Image Recognition*. The origin of the residual connection now present in every transformer.
- Kingma and Ba (2014), *Adam: A Method for Stochastic Optimization*.
- Karpathy, *The spelled-out intro to neural networks and backpropagation*. The best practical companion to section 3.9; builds an autodiff engine from nothing, which is Challenge Lab 1.
- Claude Platform documentation, *Models overview* — for the current model tiers and context limits referenced throughout.

Where to go next in this book.

Chapter 4 assembles everything from this chapter into the transformer: the same MLP layers, the same residual connections, the same layer normalisation, the same softmax output — plus one new mechanism, self-attention, which is what allows every position in a sequence to consult every other. Chapter 5 then examines attention in detail and explains why decoder-only models came to dominate. After that, Chapter 6 makes tokens and embeddings concrete, and Part 1 closes with training, alignment and safety.